

ASSIGNMENT-10.3

2303A51595

B-10

TASK-1

Prompt :

Check the following Java code for factorial and identify the logical error. Explain the problem and give the corrected version of the code.

Code :

// Original Code (Given Code)

```
public class FactorialTest {
    static int factorial(int n) {
        int result = 1;
        for (int i = 1; i < n; i++) {
            result = result * i;
        }
        return result;
    }
    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}
```

// Corrected Code

```
public class FactorialTest {

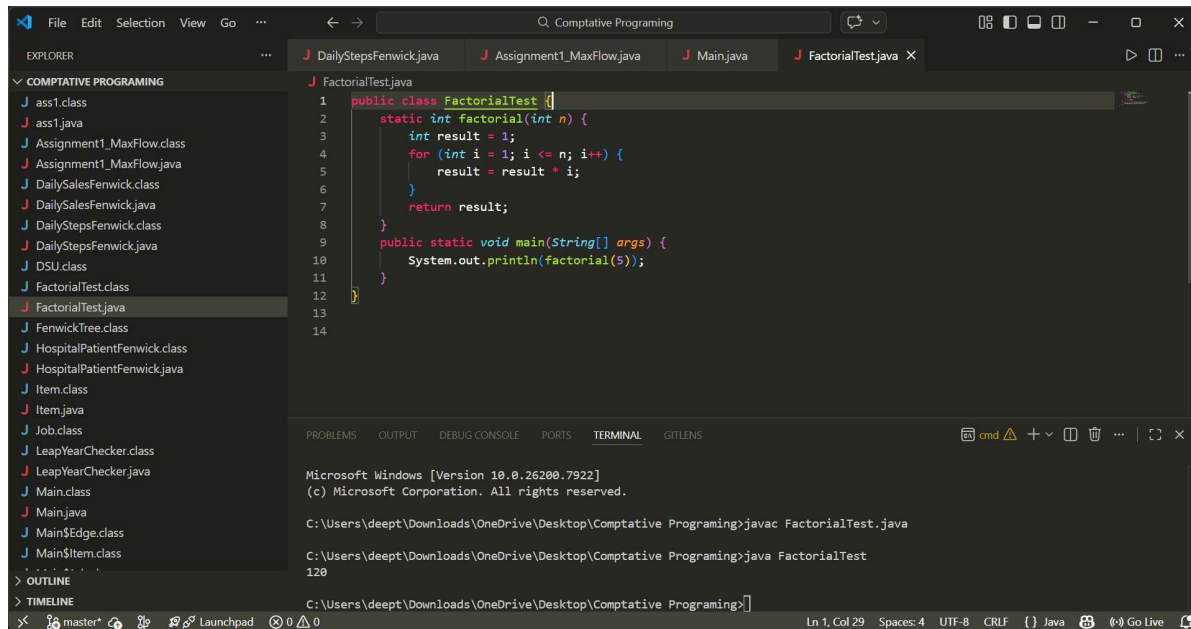
    static int factorial(int n) {
        int result = 1;

        for (int i = 1; i <= n; i++) {
            result = result * i;
        }

        return result;
    }

    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}
```

OUTPUT :



```
1 public class FactorialTest {
2     static int factorial(int n) {
3         int result = 1;
4         for (int i = 1; i < n; i++) {
5             result = result * i;
6         }
7         return result;
8     }
9     public static void main(String[] args) {
10        System.out.println(factorial(5));
11    }
12 }
13
14
```

Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\deepthi\Downloads\OneDrive\Desktop\Comptative Programming>javac FactorialTest.java

C:\Users\deepthi\Downloads\OneDrive\Desktop\Comptative Programming>java FactorialTest

120

C:\Users\deepthi\Downloads\OneDrive\Desktop\Comptative Programming>

Explanation :

The bug occurs because the loop condition uses $i < n$, so the loop stops at $n-1$ and does not multiply the number n . Because of this, the factorial calculation becomes incorrect. Changing the loop to $i \leq n$ fixes the problem and gives the correct factorial value.

Problem Statement 2 – Improving Readability & Documentation

Prompt :

Review the following Java function and identify readability issues. Rewrite the code with better names, documentation, and error handling.

Code : public class Calculator {

```
/**
 * Performs basic arithmetic operations on two numbers.
 *
 * @param num1 First numbe
 *
 * @return result of the operation
 */
static double calculateNumbers(double num1, double num2, String
operation) {
```

```

    if (operation == null) {
        throw new IllegalArgumentException("Operation cannot be null");
    }
    switch (operation) {
        case "add":
            return num1 + num2;
        case "sub":
            return num1 - num2;
        case "mul":
            return num1 * num2;
        case "div":
            if (num2 == 0) {
                throw new ArithmeticException("Division by zero not
allowed");
            }
            return num1 / num2;
        default:
            throw new IllegalArgumentException("Invalid operation");
    }
}

public static void main(String[] args) {
    System.out.println(calculateNumbers(5, 3, "add"));
    System.out.println(calculateNumbers(10, 2, "div"));
}
}

```

Output :

The screenshot shows an IDE with the following components:

- EXPLORER:** A list of files including `ass1.class`, `ass1.java`, `Assignment1_MaxFlow.class`, `Assignment1_MaxFlow.java`, `Calculator.class`, `Calculator.java`, `DailySalesFenwick.class`, `DailySalesFenwick.java`, `DailyStepsFenwick.class`, `DailyStepsFenwick.java`, `DSU.class`, `FactorialTest.class`, `FenwickTree.class`, `HospitalPatientFenwick.class`, `HospitalPatientFenwick.java`, `Item.class`, `Item.java`, `Job.class`, `LeapYearChecker.class`, `LeapYearChecker.java`, `Main.class`, `Main.java`, `Main$Edge.class`, and `Main$Edge.java`.
- EDITOR:** Displays the `Calculator.java` file. The code is as follows:


```

1 public class Calculator {
2
3     /**
4      * Performs basic arithmetic operations on two numbers.
5      *
6      * @param num1 First number
7      * @param num2 Second number
8      * @param operation Operation type (add, sub, mul, div)
9      * @return result of the operation
10    */
11    static double calculateNumbers(double num1, double num2, String operation) {
12
13        if (operation == null) {
14            throw new IllegalArgumentException("Operation cannot be null");
15        }
16
17        switch (operation) {
18

```
- TERMINAL:** Shows the execution of the program. The command `C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>javac Calculator.java` is executed, followed by `C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>java Calculator`, which produces the output `8.0` and `5.0`.

Explanation :

The improved version uses **descriptive names, proper documentation, and exception handling**. This makes the code **more readable, maintainable, and robust**.

Problem Statement 3 – Enforcing Coding Standards (Java)

Prompt : Check the following Java code for coding standard violations. Refactor the code using proper naming, spacing, and formatting.

Code :

Original Code (Given Code)

```
public class PrimeTest {
    static boolean Checkprime(int n){
        for(int i=2;i<n;i++){
            if(n%i==0){
                return false;
            }
            return true;
        }
        public static void main(String[] args){
            System.out.println(Checkprime(7));
            System.out.println(Checkprime(10));
        }
    }
}
```

Improved Java Code (Standard Version)

```
public class PrimeTest {
    static boolean checkPrime(int n) {
        for (int i = 2; i < n; i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }
    public static void main(String[] args) {
        System.out.println(checkPrime(7));
    }
}
```

```

        System.out.println(checkPrime(10));
    }
}

```

Output :

```

// PrimeTest.java
public class PrimeTest {
    static boolean checkPrime(int n) {
        for (int i = 2; i < n; i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(checkPrime(7));
        System.out.println(checkPrime(10));
    }
}

```

```

5.0
C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>javac PrimeTest.java
C:\Users\deep\Downloads\OneDrive\Desktop\Comptative Programing>java PrimeTest
true
false

```

Explanation :

AI tools help developers automatically detect coding issues such as **naming problems, formatting errors, and readability issues**.

This helps teams maintain consistent coding standards and improves overall code quality. Using AI for code reviews saves time and makes development faster in large software projects.

Problem Statement 4 – AI as a Code Reviewer in Real Projects

Prompt :

Review the following Java function and suggest improvements for readability and validation. Rewrite the function with better naming and a configurable multiplier.

Code :

```

import java.util.*;

public class DataExample {

    static List<Integer> doubleEvenNumbers(List<Integer> numbers, int multiplier) {

        if (numbers == null) {
            throw new IllegalArgumentException("Input list cannot be null");
        }
    }
}

```

```

    }

    List<Integer> result = new ArrayList<>();

    for (Integer num : numbers) {

        if (num != null && num % 2 == 0) {
            result.add(num * multiplier);
        }

    }

    return result;
}

public static void main(String[] args) {
    List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 6);
    System.out.println(doubleEvenNumbers(numbers, 2));
}
}

```

Output :

The screenshot shows an IDE with the following components:

- EXPLORER:** A list of files including `Comptative Programming`, `DataExample.class`, `DataExample.java`, `DSU.class`, `FactorialTest.class`, `FenwickTree.class`, `HospitalPatientFenwick.class`, `HospitalPatientFenwick.java`, `Item.class`, `Item.java`, `Job.class`, `LeapYearChecker.class`, `LeapYearChecker.java`, `Main.class`, `Main.java`, `Main$Edge.class`, `Main$Item.class`, `Main$Job.class`, `Main$Pair.class`, `Meeting.class`, `PrimeTest.class`, `RainfallFenwick.class`, `RainfallFenwick.java`, and `SegmentTreeLazyMin.class`.
- EDITOR:** The `DataExample.java` file is open, showing the following code:


```

1  import java.util.*;
2
3  public class DataExample {
4
5      static List<Integer> doubleEvenNumbers(List<Integer> numbers, int multiplier) {
6
7          if (numbers == null) {
8              throw new IllegalArgumentException("Input list cannot be null");
9          }
10
11          List<Integer> result = new ArrayList<>();
12
13          for (Integer num : numbers) {
14
15              if (num != null && num % 2 == 0) {
16                  result.add(num * multiplier);
17              }
18          }
19      }
20  }

```
- TERMINAL:** The output of the program is shown:


```

true
false

C:\Users\deeph\Downloads\OneDrive\Desktop\Comptative Programming>javac DataExample.java
C:\Users\deeph\Downloads\OneDrive\Desktop\Comptative Programming>java DataExample
[4, 8, 12]

```

Explanation :

AI tools can help developers quickly identify problems in code such as poor naming, missing validation, and readability issues. However, AI should be used as an assistant for code review, while human developers make the final decisions.

